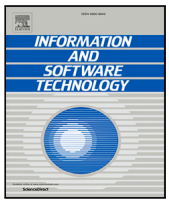
Information and Software Technology 200 (2026) 108307



Contents lists available at ScienceDirect

Information and Software Technology

journal homepage: www.elsevier.com/locate/infsof



AgentOps Pattern Catalogue: Architectural patterns for safe and observable operations of foundation model-based agents☆

Boming Xia [a,b,c,*], Yue Liu [d], Qinghua Lu [c,e,a], Liming Zhu [c,e], Dino Sejdinovic [a,b], Xiwei Xu [c,e]

[a] Responsible AI Research (RAIR) Centre, Adelaide, SA, Australia
[b] Adelaide University, Adelaide, SA, Australia
[c] CSIRO, Sydney, NSW, Australia
[d] Australian National University, Canberra, ACT, Australia
[e] University of New South Wales, Sydney, NSW, Australia

ARTICLE INFO

Keywords:
Agent
Agentic
AgentOps
Large language model
LLMs
Foundation model
Pattern
Architectural pattern
Observability

ABSTRACT

Context: Foundation model-based agents execute multi-step, tool-mediated trajectories whose behaviour is stochastic, path-dependent, and coupled to mutable state and external systems. Production operation therefore requires in-loop decision points at step boundaries and evidence handling that supports attributable, trajectory-scoped review beyond ad hoc logs.
Objectives: Provide reusable, platform-level architectural guidance for operating tool-using, stateful agents post-deployment.
Methods: We conducted a multivocal literature review and synthesised recurring operational problems and solution structures into an architectural pattern catalogue using a consistent pattern template.
Results: We present an AgentOps Architectural Pattern Catalogue of ten platform-level patterns. Each pattern specifies context, problem, forces, solution, consequences, related patterns, and documented known uses. We map the catalogue's consequences onto the ISO/IEC 25010 quality characteristics and label each pattern's implementation-evidence maturity. We also provide stakeholder-oriented adoption guidance for different roles.
Conclusion: The catalogue offers a structured basis for designing AgentOps platforms with explicit trajectory-scoped decision boundaries and evidence-backed oversight, supporting safe and observable operational practice.

1. Introduction

Foundation model-based agents (FM agents) execute multi-step *runs/trajectories* that interleave model inference with external actions (e.g., tool or API calls) and evolving state [1,2]. Unlike request/response services, runs are often stochastic and path-dependent: the same initial intent can yield different intermediate decisions, action sequences, and downstream effects due to probabilistic decoding, non-deterministic tool and environment I/O, mutable retrieval or memory state, and evolving operational constraints or system versions [3,4]. In production, correctness and risk therefore depend on sequences of decisions and actions within a run, particularly when external actions can induce irreversible side effects and durable state is carried across steps and turns [5,6]. Consider, for illustration, an agent that retries a side-effecting tool call after a timeout. The external system is mutated twice, while request-scoped logs record only two well-formed API calls. Neither the trajectory that produced them nor the policy configuration in force at the time is recoverable.

AgentOps denotes the engineering discipline for operating and evolving FM agents in real environments under uncertainty, with emphasis on post-deployment monitoring, control, evaluation, and review of tool-using, stateful runs [6–8]. This paper focuses on post-deployment operation and change governance, and assumes off-the-shelf FMs rather than retraining or fine-tuning. Development-time practices remain essential, but are not sufficient when consequential failures and harms can materialise at runtime through tool-mediated side effects and coupling with mutable state and external dependencies [9].

☆ This article is part of a Special issue entitled: 'GenAI SS' published in Information and Software Technology.

* Corresponding author at: Adelaide University, Adelaide, SA, Australia.
E-mail addresses: boming.xia@adelaide.edu.au (B. Xia), yue.liu1@anu.edu.au (Y. Liu), qinghua.lu@csiro.au (Q. Lu), liming.zhu@csiro.au (L. Zhu), dino.sejdinovic@adelaide.edu.au (D. Sejdinovic), xiwei.xu@data61.csiro.au (X. Xu).

https://doi.org/10.1016/j.infsof.2026.108307
Received 12 January 2026; Received in revised form 28 July 2026; Accepted 24 August 2026
Available online 1 September 2026
0950-5849/© 2026 The Authors. Published by Elsevier B.V. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

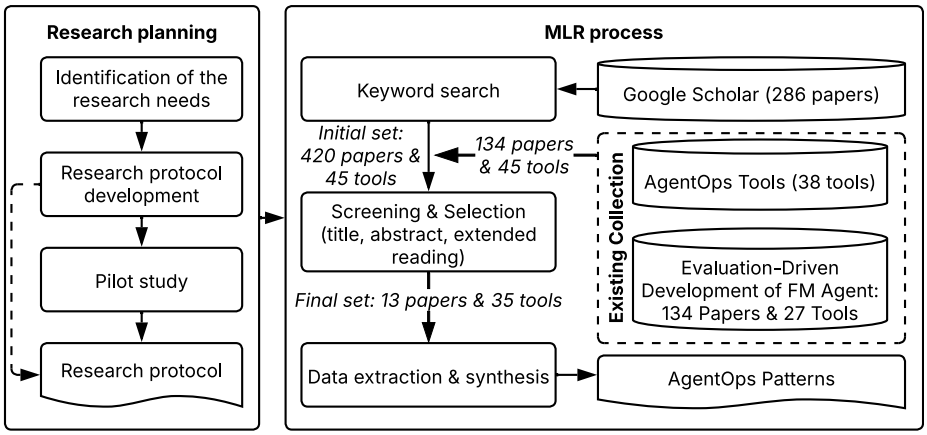


Fig. 1. Methodology overview. Anchor collections and an academic search feeding two-stage screening to the final corpus of 13 academic and 35 grey sources.

Existing operational paradigms provide important foundations but do not consistently foreground trajectory-scoped operation of tool-using agents. DevOps [10] targets reliable delivery and operation of services, while MLOps [11] addresses model-centric pipelines such as data processing, training, and deployment. LLMOps has emerged as an umbrella term for operating LLM-enabled applications, spanning prompt and retrieval asset management, evaluation, monitoring, and deployment workflows [12]. In practice, these workflows are often organised around per-inference interactions and artefacts, which leaves trajectory-scoped decision points and evidence bindings to ad hoc integration when agents are introduced [9,13]. FM agents, by contrast, require recurring in-loop decision points at step boundaries, especially where side effects or durable state commits may occur, and evidence handling that supports attribution and review across an entire run rather than isolated calls.

To make these responsibilities explicit for platform design, we adopt a viewpoint that separates recurring concerns by interaction contract. We distinguish the *execution plane*, the on-path runtime where runs execute under context and state coupling and where external actions may produce side effects, from a *control plane* that governs in-loop decisions and enforcement such as policy checks, tool authorisation, escalation, autonomy modes, and release gating [14–16]. We also distinguish an *evidence plane* that captures and organises artefacts needed for attributable review and change governance, including trajectory traces, evaluation results, configuration references, and decision records. The three planes reflect a separation of concerns and interaction contracts rather than a required physical decomposition into separate services.

We present an **AgentOps Architectural Pattern Catalogue** for designing and building AgentOps platforms that operate FM agents in production. The catalogue specifies ten reusable, platform-level building blocks and describes each pattern using a consistent template (context, problem, forces, solution, consequences, known uses, and related patterns). We focus on platform mechanisms needed to execute, control, and evidence tool-mediated, stateful runs at runtime, and exclude agent-internal design techniques (e.g., prompting strategies and planner algorithms). Each pattern is grounded with documented known uses from academic and industrial sources. We provide an overview interaction map (Fig. 2) that summarises typical composition relationships across the three planes, and stakeholder-oriented adoption guidance for engineering, evaluation/safety, and governance/compliance audiences.

The remainder of the paper is organised as follows. Section 2 details the research methodology. Section 3 presents the ten patterns. Section 4 discusses consequence trade-offs and implementation maturity across the catalogue and gives adoption guidance by audience. Section 5 positions the catalogue against related work, Section 6 states limitations, and Section 7 concludes with the main takeaway.

2. Methodology

We derived the AgentOps Architectural Pattern Catalogue via a structured multivocal literature review (MLR) following established guidelines [17,18]. An MLR is appropriate here because many post-deployment AgentOps mechanisms are specified in practitioner tools and platform documentation, and relying on academic literature alone would miss substantial operational practice. The objective was to identify and synthesise recurring *platform-level architectural mechanisms* for post-deployment operation of FM agents in production, not to measure effectiveness. We scoped to runtime mechanisms, governance interfaces, and evidence-binding infrastructure, and excluded model training and agent-internal prompt or planner design. Fig. 1 summarises the workflow and screening counts.

2.1. Corpus construction & search

Because AgentOps mechanisms are reported across both peer-reviewed literature and practitioner documentation, we constructed the corpus using two anchor reviews and a targeted academic extension.

We anchored on (i) a multivocal review of AgentOps tooling [7] (38 observability tools) and (ii) a structured review on evaluation-driven development for LLM agents [19] (134 papers; 27 tools). After merging and de-duplication, this anchor pool comprised 134 papers and 45 tools.

To broaden peer-reviewed coverage of operational mechanisms, we added one academic search round (2022 onward) using Google Scholar with the query in December 2025: `AgentOps OR (allintitle:(Agent OR Agentic) AND (observability OR monitoring OR monitor OR operation OR operations))`. This returned 286 candidate papers. We did not run an additional grey-literature search because the AgentOps tooling review already provides broad practitioner coverage for operational tools.

2.2. Screening and inclusion criteria

We screened sources in two stages (title and abstract, then full text), removing duplicates before full-text review. A source was included if it:

- addressed post-deployment operation, governance, evaluation, or assurance of FM agents;
- described a system- or platform-level mechanism with interacting components or interfaces; and
- provided sufficient technical detail to extract a problem-solution structure for pattern synthesis or to substantiate a known use.

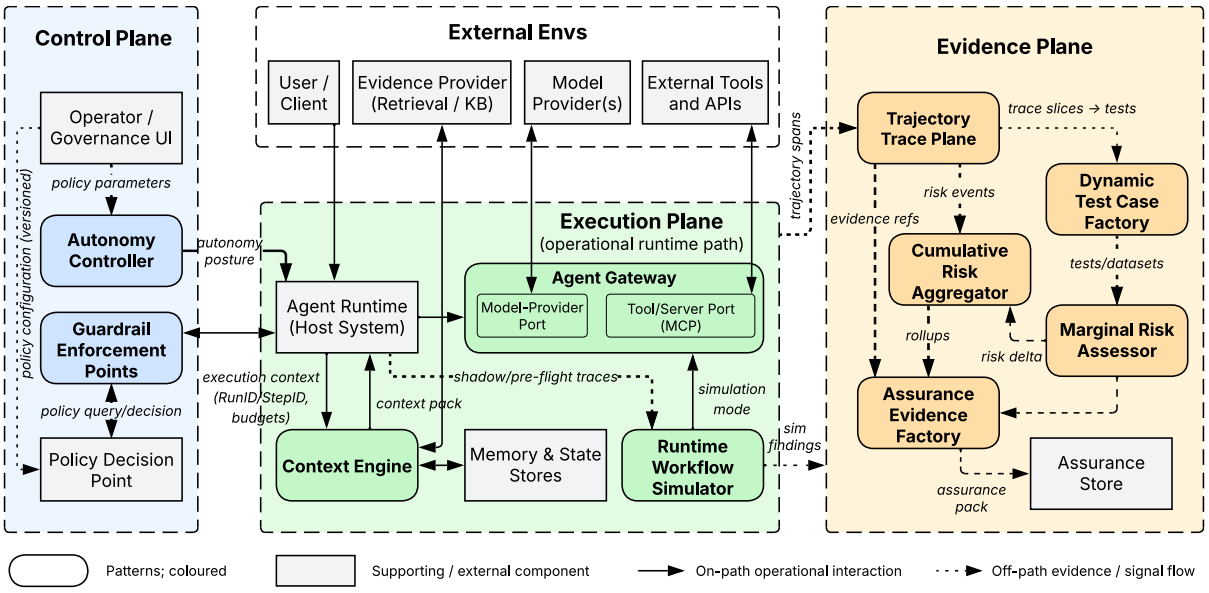


Fig. 2. Catalogue interaction map (AgentOps viewpoint). The ten patterns and supporting components across the execution, control, and evidence planes, with on-path and off-path interactions distinguished per the legend.

We excluded sources focused solely on agent-internal algorithms, development-time benchmarking without an operational systems dimension, multi-agent coordination as the primary contribution, or vendor material lacking technical specificity. The study assumes a post-deployment operational scope, the use of off-the-shelf FMs rather than model training, and platform-level architectural granularity that excludes agent-internal prompting, planning, and retrieval techniques.

After full-text screening, the final synthesis corpus comprised 13 academic sources and 35 grey sources or tools. The academic subset is smaller because most agent publications focus on agent-internal methods or development-time benchmarking, whereas production operational mechanisms are frequently specified in tool and platform documentation.

2.3. Pattern synthesis and specification

We synthesised the catalogue through qualitative coding of the included sources. We first coded recurring operational problems and mechanisms, then clustered related codes into candidate patterns. We retained only mechanisms that could be expressed as reusable platform-level building blocks with clear boundaries and explicit cross-component contracts. The synthesis relied on recurrence and architectural coherence rather than source-frequency thresholds.

Each retained pattern is reported using a consistent template (*Context*, *Problem*, *Forces*, *Solution*, *Consequences*, *Known Uses*, and *Related Patterns*). The derivation corpus was used to identify recurring operational problems and solution structures. Documented Known Uses were substantiated separately through targeted verification of vendor, framework, and standards documentation. Known Uses provide evidence that a mechanism has been implemented, but were not treated as additional frequency evidence for deriving the catalogue.

To support cross-pattern comparison, we subsequently mapped the consequences of the final ten pattern specifications to the ISO/IEC 25010:2023 product-quality characteristics and assigned implementation-evidence maturity based on the documented Known Uses. These are catalogue-level syntheses rather than frequency analyses of the MLR corpus.

3. AgentOps architectural pattern catalogue

This section presents the *AgentOps Architectural Pattern Catalogue*: ten reusable, platform-level architectural patterns for operating foundation-model agents in production. We use *AgentOps* to denote post-deployment operational mechanisms for monitoring, control, evaluation, and assurance of tool-using, stateful agent trajectories, including on-path mediation at step boundaries where actions may cause external side effects or commit durable state. Each entry is specified as an instantiable subsystem or interface boundary with an explicit cross-component interaction contract and decision points, emphasising platform responsibilities (interfaces, composition, and evidence linkage) rather than agent-internal reasoning, planning, or capability techniques. A pattern may be realised as a shared service or as a platform-provided library co-located with the runtime; the defining feature is the platform-governed contract.

Organisation. Fig. 2 provides a catalogue interaction map that locates patterns and summarises common information dependencies across three planes: *execution* (trajectory execution), *control* (in-loop governance and enforcement), and *evidence* (trace, evaluation, and assurance artefacts). The map is a navigational viewpoint, not a prescriptive reference architecture: deployments typically instantiate a subset of patterns and may realise dependencies through different boundaries and trust domains. Additional relationships are captured per pattern in *Related Patterns*. Table 1 summarises placement and the AgentOps-specific differentiator relative to request-scoped LLMOps. Consequence trade-offs across the catalogue are summarised in Table 2 and discussed in Section 4.1.

3.1. Execution plane patterns

3.1.1. Runtime workflow simulator

Summary: A *Runtime Workflow Simulator* (re-)executes or mirrors an agent *run graph* (a trajectory of step-scoped model calls, tool invocations, and state transitions) in a *controlled execution environment* (Fig. 3).

Context: Step-level guardrails can reduce local failures, but they are not sufficient to establish end-to-end safety, compliance, or operational correctness in long-horizon agent runs. Hazards often emerge at the

Table 1
Catalogue overview. Plane placement and agent-specific differentiator per pattern.

Pattern	Placement	Agent-specific differentiator (why not LLMOps)
Runtime Workflow Simulator	Execution	Evaluates multi-step workflows with tool interactions, not isolated model calls.
Agent Gateway	Execution	Governs tool and external dependency invocation with side effects, not just LLM APIs.
Context Engine	Execution	Builds step-scoped context for evolving agent trajectories, not request-scoped prompts.
Guardrail Enforcement Points	Control	Enforces policy inside agent trajectories (pre-effect, pre-persistence), not only at I/O.
Autonomy Controller	Control	Controls autonomy at step boundaries that can cause side effects or commit state.
Trajectory Trace Plane	Evidence	Traces agent runs as step- and tool-scoped trajectories, not request logs.
Cumulative Risk Aggregator	Evidence	Aggregates risk across trajectories, not per-response scores.
Dynamic Test Case Factory	Evidence	Derives replayable multi-step regression cases from production runs, not static prompts.
Marginal Risk Assessor	Evidence	Compares candidate vs baseline at trajectory scope, not standalone benchmarks.
Assurance Evidence Factory	Evidence	Packages claim-scoped assurance evidence from runtime artefacts, not ad hoc logs.

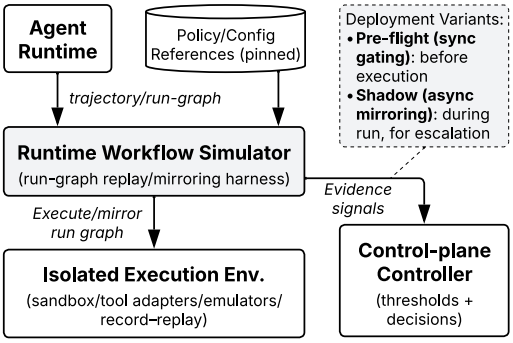


Fig. 3. Runtime Workflow Simulator. Replay and mirroring harness, isolated execution environment, and control-plane hand-off, with pre-flight and shadow variants.

trajectory level: retries and branching can amplify a single miss; partial failures can trigger duplicated tool actions that mutate external state; and cross-step carryover (summaries, injected snippets, or stale state) can bias downstream decisions. These risks arise *within* the run graph through tool-coupled side effects and cross-step state evolution rather than only at input and output boundaries [20]. In addition, debugging and assurance are complicated by model stochasticity and external service variability, motivating record-and-replay and simulation-first approaches for agent workflows [21].

Problem: How can an AgentOps platform assess an agent's run graph for trajectory-level hazards and regressions, without risking real side effects or exposing production data?

Forces:

- **Side-effect containment vs behavioural realism:** Assessment should avoid mutating real external state or exposing sensitive data, but reduced realism can miss hazards driven by true tool semantics and cross-step state evolution.
- **Evidence comparability vs execution variability:** Release decisions need comparable signals across runs and versions, but model stochasticity and changing tool timing or environments reduce repeatability and confound attribution.
- **Fidelity vs operational cost:** Higher-fidelity simulation can reveal more trajectory hazards, but increases compute, latency, and storage, limiting how broadly and frequently assessment can run.

Solution: Introduce a *Runtime Workflow Simulator* as an orchestration-and-comparison harness that consumes (i) the agent's *run graph* (including tool-coupled steps and state evolution) and (ii) *pinned configuration references* needed for comparable assessment (e.g., policy versions and tool interface contracts). The simulator executes, or mirrors, the run graph through a *controlled execution environment* designed to reduce real-world side effects, using safe tool adapters and controlled providers (e.g., stubs, emulators, or replay stores) as appropriate to the workflow. It produces *evidence signals* (e.g., detected violations, drift indicators) and delivers them to a designated *control-plane controller* that owns thresholds and the final allow, block, or escalation decision, keeping the simulator focused on evidence production rather than enforcement.

Two deployment variants reuse the same architectural core. In **pre-flight simulation** (synchronous gating), the simulator runs before privileged execution and emits evidence to the control plane for a gating decision. In **shadow simulation** (asynchronous mirroring), the simulator runs alongside the live trajectory without adding user-visible latency and emits drift or violation evidence for monitoring, audit, and escalation workflows.

Consequences - Benefits:

- **Side-effect containment:** Enables trajectory-level assessment with reduced risk of mutating real external state or handling sensitive production data.
- **Behavioural realism:** Improves detection of hazards that depend on tool semantics and cross-step state evolution.
- **Evidence comparability:** Produces run-scoped signals that make regressions easier to distinguish from execution variability across runs and releases.

Consequences - Drawbacks:

- **Behavioural realism maintenance:** Keeping simulated semantics aligned with evolving tools, dependencies, and policies adds ongoing engineering and operational overhead.
- **Comparability under mismatch:** If simulated and production semantics diverge, evidence can degrade through missed hazards or spurious violations, weakening decision confidence.
- **Evidence sensitivity:** Higher fidelity can expand the sensitive evidence footprint requiring stricter governance.

Known Uses:

- **Vertex AI Agent Engine Code Execution (managed sandbox):** provides a secure, isolated code-execution sandbox for agents; a controlled execution substrate that can be used to reduce side-effect risk during workflow assessment.[1]
- **AWS Prescriptive Guidance (simulation and test-bed agents):** describes simulation/test-bed agents for structured exploration prior to production deployment in controlled environments.[2]
- **AgentRR (Agent Record & Replay)** [21]: proposes record-and-replay for agent interaction traces and structured replayed "experience" to support safer and more efficient agent execution.
- **MCPWorld (containerised, instrumented agent testbed)** [22]: provides an isolated, instrumented environment for evaluating tool- and UI-coupled agent workflows with repeatable task setups.[3]

[1] https://docs.cloud.google.com/agent-builder/agent-engine/code-execution/overview

[2] https://docs.aws.amazon.com/prescriptive-guidance/latest/agentic-ai-patterns/simulation-and-test-bed-agents.html

[3] https://github.com/SAAgent/MCPWorld

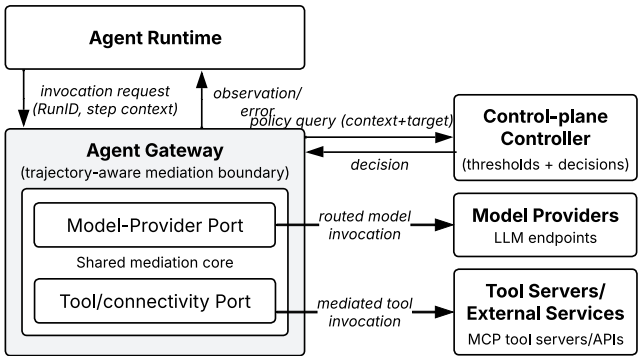


Fig. 4. Agent Gateway. Trajectory-aware mediation boundary between the agent runtime, model providers, and tool endpoints.

- **BAML VCR (LLM call record & replay for tests):** records and replays LLM-facing function calls to avoid live API calls during tests; it does not, by itself, simulate tool side effects or external state evolution.[4]

Related Patterns:

- **Agent Gateway:** Mediates tool invocation and can route simulation runs to safe tool adapters or replay stores while preserving run and step identity.
- **Context Engine:** Ensures simulated and production runs use consistent context-assembly contracts and configuration references for comparability.
- **Guardrail Enforcement Points:** Reuses identical enforcement surfaces in simulation and production to validate workflow-level governance.
- **Trajectory Trace Plane:** Supplies the run graph for replay or mirroring and binds simulator evidence (deltas, violation traces) to trajectory identity and configuration references.
- **Cumulative Risk Aggregator:** Aggregates trajectory-level violations and drift findings over time to support decision-grade oversight.
- **Dynamic Test Case Factory:** Derives regression scenarios and test templates from labelled simulator evidence and traced trajectories.

3.1.2. Agent gateway

Summary: An *Agent Gateway* is a connectivity boundary that mediates an agent's *external dependencies*, including *model-provider calls* and *tool/connectivity calls*, by enforcing configured pre-invocation controls and routing, reducing control-logic scattering across agent runtimes and integrations (Fig. 4).

Context: AgentOps platforms must govern not only what an agent *says*, but what it *does* across multi-step trajectories. In practice, agents depend on two external-facing execution surfaces: (i) *model-provider calls*, often fronted by gateway patterns to centralise authentication, quotas, routing, and observability, and (ii) *tool/connectivity calls*, which expose external systems (APIs, services, workflows, or data sources) for agent invocation, including via protocols such as the Model Context Protocol (MCP).[5] Treating these dependencies as ordinary request-scoped integrations is often insufficient for agentic execution because key hazards arise *within* the run graph: retries, branching, partial failures, and cross-step state evolution can amplify side effects and corrupt external state if not governed at invocation time. Without a dedicated gateway boundary, platforms may duplicate policy logic across executors and adapters, or rely on generic API gateways that do not carry agent-specific governance context needed at invocation time (e.g., side-effect sensitivity and operational posture).

Problem: How can an AgentOps platform centralise pre-invocation mediation for both model-provider and tool/connectivity calls, while avoiding policy scattering across agent runtimes and dependency integrations?

Forces:

- **Trajectory-conditioned control vs context-poor invocations:** Invocation risk depends on run-time context (e.g., autonomy posture), but many dependency interfaces are request-scoped and do not carry or enforce that context.
- **Uniform mediation vs dependency heterogeneity and churn:** Platforms want consistent access, routing, and accountability semantics across model providers and tools, but protocols, error modes, and behaviours differ and change frequently.
- **Less scattered control logic vs gateway bottleneck risk:** Consolidating mediation avoids duplicated logic across runtimes and integrations, but concentrates throughput and failure modes, increasing bottleneck and blast-radius exposure.

Solution: Introduce an *Agent Gateway* as the platform's external-dependency mediation boundary, positioned between the agent runtime and external executable dependencies. Architecturally, the gateway exposes two ports that share a common mediation core: a **model-provider port** and a **tool/connectivity port** (e.g., via MCP). Deployments may implement either port independently or both under a single gateway boundary.

For **model-provider calls**, the gateway centralises provider selection and operational controls (e.g., authentication, quotas, and routing) so that access governance is applied consistently at the point of invocation. For **tool/connectivity calls**, the gateway mediates protocol and access, applies pre-effect authorisation and routing (e.g., routing to a safe adapter when required). The gateway applies controls configured by the platform control plane but does not implement planning, prompting, or evaluation logic, and it does not own final escalation decisions. Its role is to provide consistent pre-invocation mediation for side-effecting external calls and to emit evidence at the boundary where external effects would otherwise occur.

Consequences - Benefits:

- **Less scattered control logic:** A single mediation boundary reduces duplicated access and routing logic across runtimes and integrations.
- **Trajectory-conditioned control:** Applying constraints and routing immediately before invocation improves control of side-effecting calls despite context-poor dependency interfaces.
- **Uniform mediation under churn:** A consistent boundary helps maintain stable governance semantics as providers, tools, and protocols evolve.

Consequences - Drawbacks:

- **Gateway bottleneck risk:** Central mediation increases blast radius and latency sensitivity under overload, outage, or misconfiguration.
- **Heterogeneity upkeep:** Keeping mediation consistent across changing protocols and behaviours adds ongoing integration and operational effort.
- **Context calibration risk:** If invocation context is coarse or incomplete, mediation can overblock or underconstrain, requiring iterative tuning from operational feedback.

Known Uses:

- **Portkey (model-provider & tool/connectivity ports):** provides an LLM gateway (routing/controls) and an MCP gateway (unified

[4] https://github.com/gr-b/baml_vcr
[5] https://modelcontextprotocol.io/

access to remote MCP servers) as complementary connectivity boundaries.[6,7]

- **agentgateway (model-provider & tool/connectivity ports):** positions a gateway connecting AI agents, MCP tool servers, and LLM providers with security, observability, resiliency, and multi-tenancy at the connectivity boundary.[8]
- **Amazon Bedrock AgentCore Gateway (tool/connec-tivity port):** a managed gateway for exposing existing services as MCP-compatible tools and connecting agents to external MCP servers for discovery and invocation.[9]
- **KrakenD MCP Gateway/MCP Server (tool/connectivity port):** documents MCP gateway proxying to external MCP servers and an MCP server mode for exposing tools via MCP, with gateway middleware (e.g., auth, observability, throttling).[10,11]
- **Microsoft GenAI gateway guidance (model-provider port):** guidance and reference architectures for deploying a gateway in front of hosted LLM endpoints to centralise authentication, observability, and invocation-time controls.[12,13]

Related Patterns:

- **Guardrail Enforcement Points:** Provide the policy checks the gateway consults for allow/deny/route decisions.
- **Autonomy Controller:** Supplies autonomy modes and escalation posture that constrain what the gateway may permit per trajectory step.
- **Cumulative Risk Aggregator:** Aggregates gateway-emitted signals and violations over trajectories to drive adaptive constraints.
- **Runtime Workflow Simulator:** Relies on the gateway to redirect model/tool invocations to isolated substrates during pre-flight or shadow simulation.
- **Trajectory Trace Plane:** Binds gateway invocation records to the run graph for auditability and replay.
- **Context Engine:** Consumes normalised observations from model/tool calls for step-scoped context assembly.

3.1.3. Context engine

Summary: A *Context Engine* is a platform-governed context-assembly boundary that assembles a *step-scoped*, *budget-aware* context package for each model call (i.e., each decision point within a trajectory), so that context budgets and assembly semantics can be governed and analysed consistently across runs and workflows (Fig. 5).

Context: FM agents execute long-horizon trajectories in which each model call consumes more than the latest user input: it must also reflect evolving task state, prior tool observations, and, where applicable, external evidence. Unlike request-scoped LLM applications, agents repeatedly feed intermediate outputs back into subsequent decisions and may mutate state that shapes later steps, all under strict context-window and cost constraints. A recurring operational failure mode is *context drift*, where instruction priority and constraint adherence degrade across steps due to uncontrolled carryover and ad hoc packing of heterogeneous material [23,24]. In multi-workflow and multi-tenant deployments, teams therefore need a predictable, governed way to assemble step-scoped context that is consistent enough for diagnosis and cost control, yet configurable per workflow or run. This motivates treating instruction assets, assembly policies, and budget controls as platform-managed configuration rather than re-implementing them inside each agent instance (e.g., reusable instruction bundles such as "skills" libraries[14]).

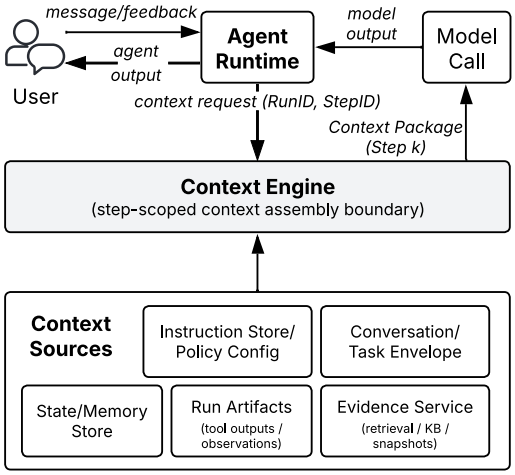


Fig. 5. Context Engine. Step-scoped assembly boundary composing context packages from platform-managed context sources.

Problem: How can an AgentOps platform provide a governable context-assembly boundary that produces relevant, budget-aware, per-step inputs across multi-step trajectories and workflows?

Forces:

- **Per-step budget vs accumulation:** Each model call has hard context and latency/cost limits, while trajectories continuously accumulate instructions, state, and tool outputs.
- **Reduction need vs reduction errors:** Selection, truncation, and summarisation are necessary, but can drop binding constraints or distort tool observations.
- **Governed consistency vs workflow variation:** Operators need consistent assembly for budgeting and diagnosis across runs and tenants, but workflows differ in what must be prioritised.

Solution: Introduce a *Context Engine* boundary between the agent runtime and a set of context sources, including static instructions, user and task input, state stores, tool-result stores, and optional external evidence providers. The Context Engine does not store memory itself; it consumes references to state, history, tool observations, and evidence maintained by separate stores. The boundary may be deployed as a shared library within the runtime or as a dedicated service, but its assembly policy and instruction assets are managed and versioned at the platform level to support consistent governance and analysis across trajectories. At each model call, the runtime requests a context package and the Context Engine composes a structured, consistently segmented input that (i) supports a consistent separation of instructions/constraints from other context material, (ii) incorporates step-relevant state and tool observations, and (iii) applies configured budget policies via explicit selection and reduction points during assembly (e.g., history selection, truncation, or summarisation) [24,25]. The Context Engine can treat retrieval as one context-source provider behind the same runtime-to-engine contract, so deployments can change retrieval strategies while keeping the assembly boundary stable.

Consequences - Benefits:

- **Budgets enforced:** Context budgeting is applied per step under a repeatable assembly contract rather than ad hoc packing.

[6] https://docs.portkey.ai/docs/product/ai-gateway
[7] https://docs.portkey.ai/docs/product/mcp-gateway
[8] https://agentgateway.dev/docs/about/introduction/
[9] https://docs.aws.amazon.com/bedrock-agentcore/latest/devguide/gateway.html
[10] https://www.krakend.io/docs/ai-gateway/mcp-gateway/
[11] https://www.krakend.io/docs/enterprise/ai-gateway/mcp-server/
[12] https://learn.microsoft.com/en-us/ai/playbook/solutions/generative-ai/genai-gateway/
[13] https://learn.microsoft.com/en-us/ai/playbook/solutions/generative-ai/genai-gateway/reference-architectures/apim-based
[14] https://code.claude.com/docs/en/skills

- **Fewer reduction errors:** Reduction behaviour is explicit and configurable, lowering silent constraint loss and observation distortion.
- **Consistent yet configurable:** Assembly is governable across runs while allowing workflow-specific priorities via configuration.

Consequences - Drawbacks:

- **Critical-path overhead:** Per-step assembly can add latency and become a throughput bottleneck.
- **Scaled misconfiguration:** Centralised reduction mistakes can repeat across many runs, increasing blast radius.
- **Change friction:** Shared assembly contracts and policies are harder to evolve without coordinated rollout.

Known Uses:

- **LangChain (step-scoped assembly hooks and state integration; context infrastructure):** documents framework surfaces (middleware and lifecycle hooks) that can modify what goes into model calls (e.g., instructions and messages) and what is persisted across steps (e.g., state/store), which can be used to realise step-scoped context assembly and reduction.[15]
- **LangGraph (pre-model hook for message selection/reduction; partial context-engine behaviour):** documents an interception surface before model calls that can be used for message selection and reduction (e.g., trimming or summarisation), supporting step-scoped control of LLM input messages.[16]
- **OpenAI Agents SDK (sessions with trimming/summarisation; partial context-engine behaviour):** provides a session boundary that rehydrates stored interaction items for subsequent runs and documents trimming/compression techniques for managing long-running interactions within context and cost budgets, supporting step- or session-scoped context management.[17,18]
- **Semantic Kernel (chat-history reduction; partial context-engine behaviour):** defines a reducer abstraction (`IChatHistoryReducer`) and provides built-in truncation and summarisation reducers that preserve system messages while reducing history size, implementing a reusable reduction component for one context source (chat history).[19]

Related Patterns:

- **Agent Gateway:** Mediates model and tool calls so context packages can be correlated with tool invocations and outcomes at the correct step.
- **Guardrail Enforcement Points:** Provides enforcement surfaces that can be invoked during context assembly and before model invocation.
- **Trajectory Trace Plane:** Binds step-scoped context artefacts or references into the run graph for replay, audits, and evaluation.
- **Autonomy Controller:** Constrains permitted context carryover and reset behaviour under the current autonomy mode.
- **Runtime Workflow Simulator:** Reuses the Context Engine boundary to reconstruct step inputs for simulation and incident reproduction.

[15] https://docs.langchain.com/oss/python/langchain/context-engineering
[16] https://langchain-ai.github.io/langgraph/how-tos/create-react-agent-manage-message-history/
[17] https://openai.github.io/openai-agents-python/sessions/
[18] https://cookbook.openai.com/examples/agents_sdk/session_memory
[19] https://learn.microsoft.com/en-us/semantic-kernel/concepts/ai-services/chat-completion/chat-history

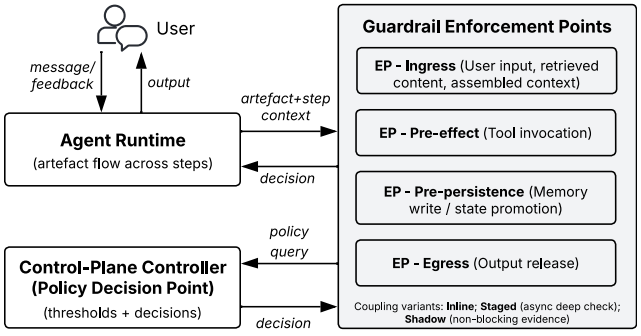


Fig. 6. Guardrail Enforcement Points. Inline policy gates at trajectory boundaries before side effects and durable state commits.

3.2. Control plane patterns

3.2.1. Guardrail enforcement points

Summary: *Guardrail Enforcement Points* are reusable enforcement interceptors deployed at selected run-graph boundaries to evaluate step-scoped agent artefacts against centrally managed policies and to apply resulting outcomes such as allow, deny, or constrain (Fig. 6).

Context: In multi-step agents, risk does not arise only at model input and output. It can emerge and compound across the run graph as artefacts evolve from user inputs to assembled context, plans, tool invocations, memory writes, and outputs [26]. Even when individual checks are effective in isolation, residual errors can propagate across steps, become embedded in persistent state, and trigger real-world side effects through tools. Request-scoped moderation or perimeter-only controls are therefore often insufficient. Policy-based access control commonly separates decision (Policy Decision Point, PDP) from enforcement (Policy Enforcement Point, PEP), with PEPs positioned at resource or service boundaries to mediate access requests [27]. In agent trajectories, enforcement needs arise at multiple step boundaries over evolving artefacts, especially where side effects occur or state is committed.

Problem: How can an AgentOps platform enforce centrally managed safety and compliance policies across multiple step boundaries in an agent trajectory?

Forces:

- **Multi-boundary coverage vs overhead and triage volume:** More enforcement boundaries reduce hazard propagation, but add latency/cost and increase outcome volume to review.
- **Uniform policy posture vs boundary-specific semantics:** Decisions should be consistent, yet the meaning of "safe" depends on which boundary is being crossed and what artefact is being judged.
- **Fail-safe gating vs workflow continuity:** Conservative blocking reduces harm potential, but can prevent legitimate progress and degrade long-horizon completion.

Solution: Introduce *Guardrail Enforcement Points* as reusable enforcement interceptors deployed at selected run-graph boundaries where artefacts are about to be used, executed, released, or committed. Each enforcement point submits the candidate artefact and boundary context to a PDP in the control plane to obtain a decision. The PDP returns an outcome such as *allow*, *deny*, or *constrain* (for example, redaction requirements or restricted execution conditions). The enforcement point applies the outcome locally and emits an enforcement outcome event with run/step identifiers and governing policy or configuration references for downstream binding and oversight.

This pattern can be instantiated at multiple boundary types, including **ingress** (user input, retrieved content, assembled context), **pre-effect** (tool/connectivity invocation gating), **pre-persistence** (memory writes and persistent state updates), and **egress** (output release).

The architectural construct remains the same: a reusable enforcement interceptor with a stable interface to the control-plane PDP.

Three architectural **variants** may be applied depending on boundary criticality. In an **inline** variant, the enforcement point runs synchronously and must return before execution proceeds. In a **shadow** variant, the enforcement point does not block the live run and instead emits evidence signals for monitoring and audits. A **staged** variant combines a lightweight inline screen with deeper asynchronous checks that may trigger control-plane escalation actions, with escalation decisions owned by the control plane.

Consequences - Benefits:

- **Multi-boundary coverage:** Earlier containment before side effects or durable commits.
- **Uniform policy posture:** Consistent decisions via a shared PDP interface across enforcement points.
- **Boundary-specific semantics:** Boundary context enables differentiated decisions without duplicating policy logic.
- **Fail-safe gating:** Inline, shadow, and staged variants let strictness match boundary criticality.

Consequences - Drawbacks:

- **Overhead and triage volume:** Additional boundaries increase runtime cost and review workload even when most runs pass.
- **Boundary-specific semantics:** Misclassified boundary context can yield wrong decisions (missed hazards or spurious blocks).
- **Workflow continuity:** Frequent interruptions can reduce end-to-end completion and user experience.

Known Uses:

- **NVIDIA NeMo Guardrails (Multi-point guardrails; inline around actions)** [28]: documents *execution rails* that trigger before and after the system invokes an action, illustrating synchronous enforcement around side-effecting steps in an agent loop.[20]
- **LangChain Guardrails (Framework interception; inline around model/tool calls):** supports middleware that intercepts execution at strategic points, including around model and tool calls, enabling synchronous boundary checks before downstream execution proceeds.[21]
- **AgentSpec (Research exemplar; multi-point enforcement in the agent loop)** [29]: hooks rule enforcement into multiple decision points (pre-action, post-observation, completion), illustrating distributed enforcement across an agent loop.
- **Strands Agents SDK (Shadow enforcement; non-blocking tuning via hooks):** documents using hooks together with Bedrock's `ApplyGuardrail` API in "shadow mode" to observe when guardrails would trigger without blocking, supporting monitoring and tuning prior to enforcement.[22]
- **Amazon Bedrock Guardrails (Ingress/egress slice; inline policy evaluation):** evaluates input prompts and model completions against configured policies during inference invocation (and can also be invoked via `ApplyGuardrail`), representing an ingress/egress enforcement slice rather than full multi-boundary run-graph enforcement.[23]

Related Patterns:

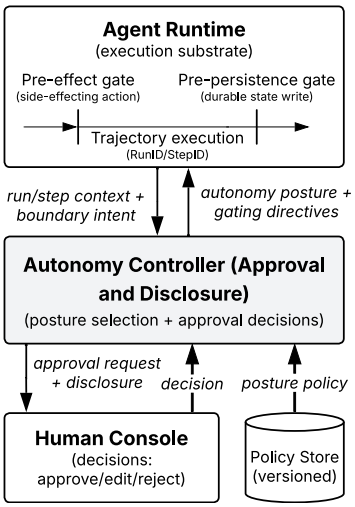


Fig. 7. Autonomy Controller. Autonomy modes and step-boundary approvals gating progression at side-effecting steps.

- **Agent Gateway:** Authoritative invocation-time mediation at the external-dependency boundary, especially for tool/connectivity calls with side effects.
- **Autonomy Controller:** Sets autonomy modes and escalation posture that constrain PDP decisions and enforcement strictness.
- **Context Engine:** Produces assembled context artefacts that are natural ingress enforcement candidates.
- **Trajectory Trace Plane:** Binds enforcement outcomes and policy versions to the run graph for replayability and audits.
- **Runtime Workflow Simulator:** Validates enforcement behaviour in pre-flight or shadow runs using the same policy posture.
- **Cumulative Risk Aggregator:** Aggregates enforcement outcomes across steps and sessions to drive adaptive constraints.

3.2.2. Autonomy controller

Summary: An *Autonomy Controller* governs an agent execution's autonomy mode by determining whether execution may proceed autonomously or requires human approval at selected step boundaries (Fig. 7).

Context: FM agents execute multi-step trajectories in which risk concentrates at boundaries that can trigger external side effects (e.g., tool invocations) or commit persistent state changes (e.g., writes to memory stores or other durable state surfaces). In operational settings, platform owners need predictable control over when autonomy is allowed or constrained, and operators need clear approval requests when intervention is required. Some deployments may also require user-facing disclosure about what the agent is permitted to do. Without an explicit control-plane mechanism, systems tend to drift towards either silent full autonomy or blanket manual approval, both of which are operationally brittle.

Problem: How can an AgentOps platform govern when an agent may act autonomously versus when it must request human approval at step boundaries, while keeping the active autonomy mode and approval requirements clear to operators (and, where needed, users)?

Forces:

- **Containment vs delay/load:** Approvals reduce harm at high-impact boundaries, but add blocking delay and can overwhelm reviewers.
- **Stable mode vs change:** Teams need a predictable autonomy mode, but incidents and policy shifts require updating it without disrupting workflows.

[20] https://docs.nvidia.com/nemo/guardrails/latest/user-guides/configuration-guide/guardrails-configuration.html

[21] https://docs.langchain.com/oss/python/langchain/guardrails

[22] https://strandsagents.com/latest/documentation/docs/user-guide/safety-security/guardrails/

[23] https://docs.aws.amazon.com/bedrock/latest/userguide/guardrails.html

- **Context vs exposure:** Approvers need decision context, but richer context increases confidentiality risk.

Solution: Introduce an *Autonomy Controller* in the control plane that maintains a configured autonomy mode for an agent execution and determines whether human approval is required before execution may proceed past selected step boundaries. The autonomy mode defines the approval posture, such as *autonomous by default*, *approval required for selected critical boundaries*, or *approval required by default*. The runtime requests a determination from the controller when it reaches a governed boundary crossing (which may be surfaced by runtime orchestration or by co-located boundary interceptors). When approval is required, the controller issues an approval request to an operator-facing channel and applies the operator's response (e.g., approve, reject, or approve with modifications/conditions) before execution proceeds. The controller records the active autonomy posture and approval outcomes as operational evidence for the execution. The Autonomy Controller governs *when* autonomy is permitted and manages approval interactions; it does not perform execution, planning, or boundary enforcement itself.

Consequences - Benefits:

- **Containment with bounded delay/load:** Checkpoints gate high-impact boundaries while making review volume and blocking time explicit and configurable.
- **Stable mode with controlled change:** A declared mode keeps approval expectations consistent while supporting controlled tightening or relaxation.
- **Context with bounded exposure:** Structured requests provide necessary context while enabling minimisation and redaction to limit exposure.

Consequences - Drawbacks:

- **Delay/load bottlenecks:** Misplaced checkpoints can flood reviewers and stall execution, reducing completion.
- **Mode-change churn:** Frequent changes reduce predictability and complicate operational coordination.
- **Context-exposure trade-off:** Over-minimised context degrades decisions; over-rich context increases exposure and review burden.

Known Uses:

- **LangGraph, LangChain, OpenAI Agents SDK, Letta (human-in-the-loop approval gating):** support pausing agent execution at defined points and resuming only after a human approve/edit/reject decision, commonly used to gate side-effecting actions (e.g., tool invocations) and other critical step boundaries.[24,25,26,27]
- **Mastra, Relevance AI Workforce (workflow-level approvals/escalations):** provide workflow approvals that suspend execution for human review and resume after approval or escalation handling, closer to workflow governance than run-graph step-boundary controllers.[28,29]
- **Replit Agent (explicit autonomy level UI):** exposes user-configurable autonomy levels as a product control surface, illustrating explicit autonomy posture but not general step-boundary approval control.[30]

[24] https://docs.langchain.com/oss/python/langgraph/interrupts
[25] https://docs.langchain.com/oss/python/langchain/human-in-the-loop
[26] https://openai.github.io/openai-agents-js/guides/human-in-the-loop/
[27] https://docs.letta.com/guides/agents/human-in-the-loop/
[28] https://mastra.ai/docs/workflows/human-in-the-loop
[29] https://relevanceai.com/approvals-escalations
[30] https://docs.replit.com/replitai/agent

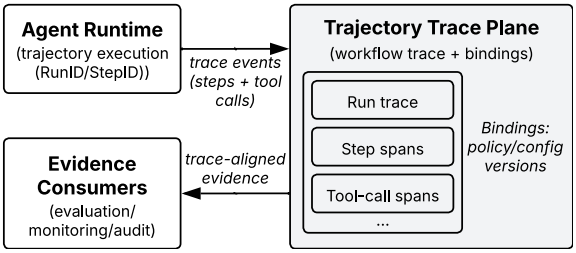


Fig. 8. Trajectory Trace Plane. Run-scoped traces with step and tool-call spans bound to policy and configuration versions.

Related Patterns:

- **Agent Gateway:** Acts as a mediation surface where approval-required actions can be gated before external side effects occur.
- **Guardrail Enforcement Points:** Can be co-located at the same boundaries to complement approval gating through policy outcomes.
- **Cumulative Risk Aggregator:** Supplies aggregated risk signals that may inform tighter autonomy modes.
- **Trajectory Trace Plane:** Binds approval requests and outcomes to the run graph as operational evidence for an agent run.
- **Runtime Workflow Simulator:** Can surface workflow-level risks that justify more conservative autonomy modes in future executions.

3.3. Evidence plane patterns

3.3.1. Trajectory trace plane

Summary: A *Trajectory Trace Plane* captures each agent run as a correlated trace with step-level events (including tool actions where instrumented) to support run-level observability and downstream governance for multi-step trajectories (Fig. 8).

Context: FM agents execute long-horizon trajectories that interleave model calls, tool invocations, branching and retries, and state mutation. Failures and harms often emerge from the interaction of steps rather than from any single model response, and post-incident analysis requires evidence that explains how actions unfolded across a run. Conventional LLMOps observability typically centres on request-scoped prompts and completions, which is insufficient to attribute tool actions and multi-step control flow across an agent execution. An AgentOps platform therefore requires a shared evidence plane that correlates step-scoped events across components and supports downstream governance, evaluation, and debugging on a consistent run representation.

Problem: How can an AgentOps platform capture attributable, step-scoped evidence for an agent run, including tool actions and multi-step control flow, without collapsing into ad hoc logs or exposing sensitive information?

Forces:

- **Granularity vs overhead:** Step- and tool-level attribution improves diagnosis, but increases telemetry volume, runtime cost, and retention burden.
- **Investigability vs minimisation:** Evidence must support review and automation, but traces can expose sensitive content, requiring minimisation, access control, and retention limits.
- **Interop vs trajectory semantics:** Standard telemetry ecosystems ease integration, but must still represent trajectory structure (steps, branching, tool actions, state commits) rather than generic request logs.

Solution: Introduce a *Trajectory Trace Plane* as an evidence-plane subsystem with a shared trace contract across the agent runtime and participating platform components. For each agent execution, the trace plane assigns or propagates a run identifier and correlates step-scoped

events emitted by participating components (e.g., step transitions and external actions) into a run-scoped trace view that can be queried and inspected for operational analysis. It supports downstream consumers (e.g., replay or simulation subsystems) by providing correlated run evidence where available.

The trace plane supports attaching lightweight, structured metadata needed for filtering and cross-system correlation (e.g., environment labels or policy/configuration reference identifiers when available), while keeping policy evaluation and configuration management outside its scope. It supports configuration of deployment-appropriate minimisation and access controls so trace capture and retention can be constrained where sensitive content is involved.

Consequences - Benefits:

- **Granularity with usable overhead:** Step-scoped correlation improves trajectory diagnosis compared to request logs, with configurable capture to manage overhead.
- **Investigability under minimisation:** Minimised capture plus access controls keeps traces reviewable while reducing sensitive exposure.
- **Interop with preserved semantics:** A shared trace contract integrates with observability tooling while preserving trajectory semantics needed for debugging and governance.

Consequences - Drawbacks:

- **Overhead pressure:** Higher granularity can force sampling, reduced retention, or dropped fields, which weakens diagnosis.
- **Sensitive store risk:** Trace stores concentrate sensitive artefacts and metadata, increasing impact if controls fail or are misconfigured.
- **Integration discipline:** Preserving interoperability and semantics requires coordinated instrumentation and schema alignment across components and tool surfaces.

Known Uses:

- **OpenAI Agents SDK, LangSmith (trace-first frameworks for agent runs; hierarchical traces with metadata/tags):** provide structured tracing for agent executions using hierarchical trace/span or run-tree representations, and support attaching metadata/tags for filtering, correlation, and provenance of traces across runs.[31,32,33]
- **AgentOps, Amazon Bedrock Agents (runtime/platform trace emission for agent execution; step-oriented trace events):** expose trace data as an operational platform feature, emitting step-oriented trace events for orchestrated agent execution (including external actions where applicable) that operators can inspect and correlate within a run.[34,35,36]
- **Langfuse, Arize Phoenix (hierarchical trace and observation model):** provide hierarchical trace and observation structures for multi-step agent executions, with identifiers and metadata for correlation and filtering across runs.[37,38]
- **OpenTelemetry GenAI semantic conventions (interoperability reference; development status):** defines draft semantic conventions for GenAI agent spans and attributes to support interoperable trace enrichment across tooling ecosystems.[39]

[31] https://openai.github.io/openai-agents-python/tracing/
[32] https://docs.langchain.com/langsmith/trace-with-langgraph
[33] https://docs.langchain.com/langsmith/add-metadata-tags
[34] https://docs.agentops.ai/v2/concepts/traces
[35] https://docs.agentops.ai/v2/usage/manual-trace-control
[36] https://docs.aws.amazon.com/bedrock/latest/userguide/trace-events.html
[37] https://langfuse.com/docs
[38] https://arize.com/docs/phoenix/tracing/llm-traces
[39] https://github.com/open-telemetry/semantic-conventions-genai

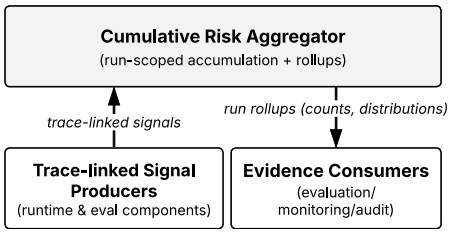


Fig. 9. Cumulative Risk Aggregator. Accumulation of trace-linked signals into run- and window-scoped risk rollups.

Related Patterns:

- **Guardrail Enforcement Points:** May emit enforcement outcomes that can be recorded as trace events.
- **Autonomy Controller:** May emit approval outcomes and autonomy mode changes that can be recorded as trace events.
- **Runtime Workflow Simulator:** Can consume trace-derived run structures for replay and workflow-level validation.
- **Cumulative Risk Aggregator:** Can aggregate trace events into decision-grade risk state across steps and sessions.
- **Dynamic Test Case Factory:** Can extract regression scenarios from trace structures and tool-call sequences where available.
- **Marginal Risk Assessor:** Can compare candidate versus baseline behaviour using trace-aligned trajectories.
- **Assurance Evidence Factory:** Can package trace-linked evidence into audit and compliance artefacts.

3.3.2. Cumulative risk aggregator

Summary: A *Cumulative Risk Aggregator* accumulates trace- or session-linked risk and evaluation signals emitted by other platform components and produces run- or session-level rollups so trajectory-level accumulation can be monitored and reviewed (Fig. 9).

Context: FM agents execute multi-step, stateful trajectories that include branching, retries, tool actions, and state mutation. Even when individual steps are guarded, residual issues can accumulate across a trajectory, for example repeated partial failures that increase exposure and escalating impact as tool side effects occur [30]. Raw traces can record what happened, but operational monitoring and oversight often require compact indicators that summarise how risk- and evaluation-related signals evolve over a *run or session*. Here, a *run* is a single trajectory execution and a *session* is a platform-defined grouping of related runs under a shared identifier (e.g., a continued task thread). This need is agent-specific because the relevant unit of analysis is a trajectory with tool actions and state coupling, not a single request/response exchange.

Problem: How can an AgentOps platform accumulate and summarise trace-linked signals across an agent run or session so trajectory-level accumulation is visible, interpretable, and reviewable without relying on ad hoc logs?

Forces:

- **Timeliness vs overhead:** Rollups must be available quickly for monitoring, but continuous accumulation must be feasible at production volume.
- **Reviewability vs minimisation:** Operators need enough context to interpret rollups, but linked evidence can expose sensitive content and patterns, requiring minimisation and access control.
- **Validity under drift vs distortion:** Indicators must remain interpretable as models, tools, and evaluators change, but aggregation can hide tail risk or amplify noise.

Solution: Introduce a *Cumulative Risk Aggregator* as an evidence-plane subsystem that ingests trace- or session-linked signals emitted

by instrumentation and evaluation components (e.g., evaluation labels/scores and risk indicators, including uncertainty-related signals when an evaluator provides them) and produces run- or session-level rollups. The pattern is defined by a well-defined boundary: an ingestion interface for trace-referenced signals and a query interface for aggregated indicators aligned to the agent trajectory scope. The aggregator does not perform primary policy decisions or generate new judgements; it summarises signals produced elsewhere.

Rollups can be lightweight (e.g., counts, rates, summary statistics, and distributions over signal values) so downstream monitoring and review can rely on compact indicators rather than repeatedly scanning full traces. Downstream actions (e.g., escalation, blocking, or reporting decisions) remain outside the aggregator's responsibility. Implementations may persist rollups as first-class artefacts or compute them from underlying stores (e.g., evaluation and trace stores).

Consequences - Benefits:

- **Timeliness with manageable overhead:** Provides low-delay run/session rollups that reduce routine full-trace scanning under load.
- **Reviewability under minimisation:** Enables monitoring from compact indicators, with controlled drill-down where permitted.
- **Validity cues under drift:** Makes aggregation behaviour visible (for example via distributions and slice views), supporting calibration when drift occurs.

Consequences - Drawbacks:

- **Overhead pressure:** Ingestion and queries add cost at scale and may force batching, sampling, or reduced granularity.
- **Minimisation-driven opacity:** Access and retention constraints can limit interpretability and increase follow-up effort to explain rollups.
- **Distortion risk under drift:** Poorly defined signals or drift can yield misleading rollups unless definitions and calibration are actively maintained.

Known Uses:

- **Langfuse Score Analytics (trace/session-linked scoring; distribution and trend analysis):** supports attaching scores to traces or sessions and analysing distributions and trends, providing primitives for trace/session-linked signal aggregation.[40]
- **Arize Phoenix Trace Evaluations (trace-linked evaluations; evaluation results logged back to traces):** supports running evaluations on captured traces and logging evaluation results back to the Phoenix UI, enabling trace-linked signal capture suitable for aggregation.[41]
- **SAUP, UProp, and CogSafe (motivating research)** [31–33]: study propagation and accumulation of uncertainty and risk signals through multi-step agentic pipelines.

Related Patterns:

- **Trajectory Trace Plane:** Provides run/session identifiers and trace linkage so signals can be correlated and aggregated at trajectory scope.
- **Guardrail Enforcement Points:** May emit enforcement outcomes that can be ingested as signals.
- **Runtime Workflow Simulator:** May produce replay and drift findings that can be accumulated as signals.
- **Autonomy Controller:** May consume aggregated indicators as inputs to approval posture decisions (policy permitting).

[40] https://langfuse.com/docs/evaluation/evaluation-methods/score-analytics

[41] https://arize.com/docs/phoenix/tracing/how-to-tracing/feedback-and-annotations/evaluating-phoenix-traces

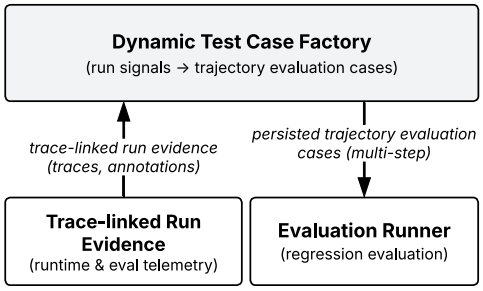


Fig. 10. Dynamic Test Case Factory. Derivation of replayable regression cases from production trajectories.

- **Marginal Risk Assessor:** May compare rollups across baselines and candidates as part of evaluation workflows.
- **Assurance Evidence Factory:** May package rollups as part of audit-oriented evidence bundles.

3.3.3. Dynamic test case factory

Summary: A *Dynamic Test Case Factory* derives *persisted regression cases* from selected *production traces and run signals* so regression evaluation can remain grounded in observed agent behaviours as workflows, tools, and models evolve (Fig. 10).

Context: FM agents operate as multi-step trajectories with branching, retries, tool calls that may have side effects, and state or context mutation across steps. Operationally significant failures and near-misses often arise from trajectory-level interactions, not from a single prompt/response exchange. Static benchmarks and one-off offline test sets can drift away from live failure modes under distribution shift, external dependency change, and vendor updates [19]. An AgentOps platform therefore benefits from a subsystem that can distil selected trace-linked operational evidence into reusable regression cases, while respecting confidentiality constraints and safe re-execution concerns.

Problem: How can an AgentOps platform distil production traces and run signals into regression cases that stay grounded in real trajectory failures and near-misses as the surrounding system evolves?

Forces:

- **Production anchoring vs re-executability:** Cases should reflect real failures, but must remain runnable as models, tools, and dependencies drift and stochasticity reduces repeatability.
- **Fidelity vs confidentiality and safe handling:** High-fidelity, multi-step reproduction is valuable, but persisting and replaying production-derived artefacts is constrained by privacy and side-effect risk.
- **Freshness vs cost and governance:** Continuous harvesting keeps coverage current, but suite growth increases evaluation cost and governance burden.

Solution: Introduce a *Dynamic Test Case Factory* as an evidence-plane subsystem that converts trace-linked operational evidence into reusable evaluation inputs. The factory consumes selected production traces and run signals, together with any available annotations (e.g., evaluator outputs, incident labels, or cohorts), and produces *persisted regression cases* that evaluation runners can re-execute across releases.

Where the underlying trace representation preserves multi-step structure, the factory may promote *trajectory-scoped* cases so regressions in ordering- and tool-sequence-dependent behaviour can be evaluated rather than only request-scoped inputs. The factory records or preserves a reference from each regression case to its originating trace evidence (e.g., trace ID or trace URL) where the underlying tooling supports it, so cases remain interpretable and maintainable as tools, prompts, and policies change.

Consequences - Benefits:

- **Production anchoring with re-executability:** Promotes observed traces into persisted cases that can be re-run across releases, keeping regression checks tied to operational failure modes.
- **Fidelity within safe, confidential bounds:** Supports trajectory-scoped cases when feasible, while enabling controlled redaction and safe replay practices.
- **Freshness with governance hooks:** Trace provenance enables review, prioritisation, and retirement as the suite evolves.

Consequences - Drawbacks:

- **Re-executability erosion:** Drift and stochasticity can make cases flaky or obsolete, increasing maintenance and retirement churn.
- **Fidelity loss from constraints:** Redaction and safe handling can reduce realism, weakening diagnostic value for some failures.
- **Cost and governance growth:** Expansion increases compute, operational complexity, and the need for tiering and access discipline.

Known Uses:

- **Langfuse (Datasets from traces; trace-to-dataset linkage):** supports creating dataset items directly from tracing observations, and supports linking traces to dataset items using a trace identifier.[42,43,44]
- **LangSmith (Backtesting; datasets from historical production traces):** describes backtesting as converting historical production logs into datasets for offline evaluation, and documents creating datasets from historical production traces as a first-class evaluation workflow.[45]
- **Agenta (Create test sets from traces):** supports promoting an observed trace into a test set from the observability view (trace-to-test-set workflow), including reviewing the mapping from trace fields to test case columns.[46]
- **Arize Phoenix (Evaluating traces via exported trace datasets):** supports exporting a trace dataset from captured traces, running evaluations, and logging evaluation results back to Phoenix, reflecting a trace-to-evaluation-asset workflow grounded in observed runs.[47]

Related Patterns:

- **Trajectory Trace Plane:** Provides trace-linked evidence and provenance used to seed candidates and bind promoted cases to their origins.
- **Guardrail Enforcement Points:** Emits violations and policy outcomes that can trigger candidate selection.
- **Runtime Workflow Simulator:** Produces replay and drift findings that can inform candidate generation and prioritisation.
- **Cumulative Risk Aggregator:** Provides run-level summaries that can help prioritise which candidates should be promoted or retained.
- **Assurance Evidence Factory:** Packages promoted cases and regression results into audit-ready evidence bundles.

3.3.4. Marginal risk assessor

Summary: A *Marginal Risk Assessor* produces a *baseline-relative risk-change report* for a candidate agent over a declared scope and time window, supporting governance decisions using explicit, comparable evidence rather than absolute risk claims (Fig. 11).

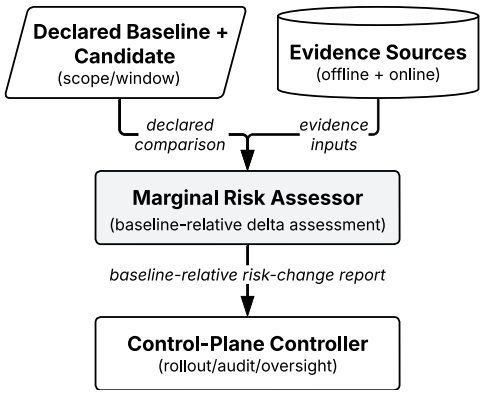


Fig. 11. Marginal Risk Assessor. Declared baseline-versus-candidate comparison over a stated scope and window.

Context: In deployment and release decisions, teams often need to decide whether a candidate system changes risk *relative to a stated baseline* (e.g., a prior release, an incumbent workflow, or access to other available resources) rather than establish an absolute risk score [34]. This comparative framing is explicit in several frontier governance approaches, including OpenAI's discussion of *marginal risk* and Amazon's use of *uplift studies* to compare outcomes with and without access to a model (see known uses). For agentic systems, baseline-relative comparison is harder because risk can shift at the *trajectory level*: multi-step execution, retries and branching, tool-mediated actions, and state mutation can change exposure and failure modes in ways that request-scoped evaluation may not reveal. Evidence is typically fragmented across offline evaluations, rollout telemetry, and incident records. Without a dedicated platform capability that binds baseline definition, declared scope/window, and evidence references into a single review artefact, marginal-risk arguments tend to become informal and difficult to review.

Problem: How can an AgentOps platform support an explicit, reviewable determination of whether deploying a candidate agent changes operational risk relative to a stated baseline for a declared scope and time window?

Forces:

- **Like-for-like vs variability:** Marginal conclusions require comparable scope and cohorts, but task mix, access paths, and coverage shift in deployment.
- **Auditability vs incompleteness:** Review needs explicit baselines and evidence lineage, but instrumentation, labels, and incident capture can be partial.
- **Decision timeliness vs uncertainty disclosure:** Decisions must be produced quickly, but confounders and limits must be stated to avoid overstated deltas.

Solution: Introduce a *Marginal Risk Assessor* as an evidence-plane component that produces a *baseline-relative risk-change report* for a candidate agent over a declared scope and time window. Here, *scope* denotes the declared slice of operation the claim covers (e.g., specified workflows or task families, a tenant/environment, and relevant capability or tool-access constraints). The assessor exposes a stable assessment interface that accepts the baseline and candidate identifiers (or descriptions), the declared scope/window, and references to available evidence sources, and returns a versioned report artefact with evidence pointers suitable for review.

For each assessment, the report records (i) the stated baseline (identifier where available, otherwise a baseline description), (ii) the scope and time window of the comparison, and (iii) references to the evidence used for the comparison, prioritising trajectory-scoped

[42] https://langfuse.com/changelog/2024-04-25-datasets-v2
[43] https://langfuse.com/docs/evaluation/overview
[44] https://langfuse.com/docs/observability/features/trace-ids-and-distributed-tracing
[45] https://docs.langchain.com/langsmith/evaluation-types
[46] https://agenta.ai/docs/evaluation/managing-test-sets/create-from-traces
[47] https://arize.com/docs/phoenix/tracing/how-to-tracing/feedback-and-annotations/evaluating-phoenix-traces

evidence when it is available and relevant. The assessor does not compute an absolute risk score. Instead, it organises available evidence around the baseline-relative claim being made and records material limitations (e.g., missing coverage, cohort mismatch, or evidence drift) that constrain the comparison. Where quantitative comparison is supported, the report may include baseline-relative deltas; otherwise it documents qualitative differences and unknowns. The report may be published as a versioned evidence artefact for downstream governance and audit workflows; final allow, hold, or escalation decisions remain outside this pattern.

Consequences - Benefits:

- **Like-for-like discipline:** Forces explicit baseline, scope, and window, strengthening comparability despite operational variability.
- **Reviewable provenance:** Packages evidence references and coverage gaps for auditability without implying complete evidence.
- **Timely, qualified reports:** Produces operationally usable reports while surfacing confounders and limits alongside any deltas.

Consequences - Drawbacks:

- **Variability sensitivity:** Residual cohort mismatch and shifting conditions can dominate signals, limiting marginal conclusions.
- **Evidence upkeep cost:** Maintaining provenance links, cohorts, and coverage statements adds recurring engineering and operational overhead.
- **Misread uncertainty:** Deltas and disclosed limits can be oversimplified or selectively interpreted in decision forums.

Known Uses:

- **OpenAI Preparedness Framework v2 (governance precedent; marginal risk framing):** explicitly defines *marginal risk* as whether a system meaningfully increases risk relative to the baseline risk in the world.[48]
- **Amazon Frontier Model Safety Framework (governance precedent; uplift studies):** defines *uplift* as a controlled comparison of outcomes with access to a frontier model versus without access, providing a concrete baseline-relative evaluation construct.[49]
- **Alaga & Chen (research; baseline choice in marginal risk)** [34]: analyses how baseline choice affects marginal-risk conclusions and motivates explicit baseline specification for verifiable claims.
- **Chen et al. (research; marginal risk without ground truth)** [35]: proposes a marginal-risk assessment framework centred on relative evaluation rather than absolute risk scoring, reinforcing the baseline-relative framing.

Related Patterns:

- **Trajectory Trace Plane:** supplies trajectory-scoped evidence and provenance references used by the assessor.
- **Cumulative Risk Aggregator:** provides run- or window-level rollups that can be compared baseline versus candidate.
- **Dynamic Test Case Factory:** curates shared trajectory cases for like-for-like offline comparison.
- **Runtime Workflow Simulator:** generates pre-flight or shadow evidence to support comparison without side effects.
- **Guardrail Enforcement Points:** contribute enforcement outcomes that may appear as comparative signals.
- **Assurance Evidence Factory:** packages marginal-risk reports into audit- and compliance-facing artefacts.

[48] https://cdn.openai.com/pdf/18a02b5d-6b67-4cec-ab64-68cdfbddebcd/preparedness-framework-v2.pdf

[49] https://assets.amazon.science/a7/7c/8bdade5c4eda9168f3dee6434fff/pc-amazon-frontier-model-safety-framework-2-7-final-2-9.pdf

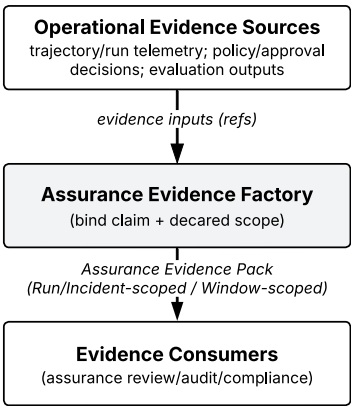


Fig. 12. Assurance Evidence Factory. Binding operational evidence to a claim and declared scope in reviewable evidence packs.

3.3.5. *Assurance evidence factory*

Summary: An *Assurance Evidence Factory* is an evidence-plane subsystem that packages scope-bounded assurance claims with reviewable references to supporting operational evidence (Fig. 12).

Context: Assurance cases are widely used in safety- and security-critical settings to present scope-bounded claims supported by assessable evidence, rather than absolute guarantees [36]. Recent frontier AI safety work similarly motivates safety-case-style arguments and ongoing reassessment as systems and mitigations evolve (e.g., [37–39]). For tool-using agents, assurance is harder because behaviour is probabilistic and trajectory-dependent: multi-step execution, retries and branching, tool-mediated side effects, and state mutation can shift risk in ways that are difficult to characterise exhaustively in advance. In practice, assurance therefore becomes conditional and evidence-driven, drawing on traces, control decisions, and evaluation results over a declared scope and review window. Without a dedicated packaging mechanism, reviewers rely on ad hoc log exports and dashboards that are difficult to interpret consistently across releases and reassessment cycles.

Problem: How can an AgentOps platform support reviewable, scope-bounded assurance claims for tool-using stochastic agents?

Forces:

- **Bounded claims vs assurance demand:** Claims must stay scoped and qualified to avoid overclaiming, yet still be decision-useful.
- **Reviewability vs sensitivity:** Review needs inspectable support, but operational evidence is sensitive and must be minimised and access-controlled.
- **Interpretability vs overhead:** References must remain resolvable across evolving models, tools, and policies, but maintaining them adds ongoing packaging overhead.

Solution: Introduce an *Assurance Evidence Factory* as an evidence-plane subsystem that produces a structured *Assurance Evidence Pack* for a stated assurance claim. Each pack associates (i) the declared claim and scope (e.g., system version, deployment context, and review window) with (ii) reviewable, resolvable references to supporting artefacts already produced elsewhere in the platform (e.g., trajectory traces, enforcement or approval records, evaluation outputs, and run- or window-level summaries).

The factory provides a packaging boundary that emits claim-scoped evidence-reference artefacts for evidence consumers (e.g., audit preparation or incident review workflows). It does not generate evidence, prescribe an assurance methodology, or make deployment decisions. Its responsibility is limited to claim-scoped packaging and explicit linkage so reviewers can see what is being claimed, under which conditions, and which evidence references the claim relies upon (subject to applicable access controls).

Consequences - Benefits:

- **Bounded claims:** Keeps claims explicitly scoped and qualified while still producing a decision-usable artefact.
- **Reviewability with sensitive evidence:** Enables review via controlled, minimised evidence references under access constraints.
- **Interpretability under evolution:** Maintains resolvable evidence links across releases and reassessment windows as upstream systems change.

Consequences - Drawbacks:

- **Assurance-demand pressure:** Stakeholder demand for clear outcomes can still incentivise oversimplification unless scope and limits are enforced.
- **Evidence-sensitivity opacity:** Minimisation and restricted access can reduce what reviewers can directly inspect, increasing reliance on process and summaries.
- **Evolution-driven overhead:** Keeping references resolvable and interpretable across tool, policy, and storage changes requires ongoing curation and refresh work.

Known Uses:

- **UK AI Security Institute safety cases (frontier AI):** *Adjacent:* argues for safety-case-style, assessable evidence packs for frontier AI development and reassessment as systems evolve.[50,51]
- **Clymer et al. (example safety case for misuse safeguards)** [37]: presents an end-to-end safety case that connects disparate evaluation and operational evidence into a deployment-relevant argument.[52]
- **OMG Structured Assurance Case Metamodel (not agent-specific):** defines a metamodel for representing assurance cases as structured claims, arguments, and evidence, providing a precedent for claim-scoped evidence organisation and exchange.[53,54]
- **ICO argument-based assurance case guidance (not agent-specific):** describes assurance cases as structured claims, arguments, and evidence, motivating disciplined evidence organisation for review.[55]

Related Patterns:

- **Trajectory Trace Plane:** provides run- and step-scoped operational artefact references.
- **Guardrail Enforcement Points:** provide enforcement outcomes that may be referenced by assurance claims.
- **Autonomy Controller:** provides approval and escalation records for accountability.
- **Cumulative Risk Aggregator:** provides windowed summaries that may be referenced in assurance packs.
- **Marginal Risk Assessor:** may reference claim-scoped packs when producing baseline-relative reports.

4. Discussion and implications

This section reads the catalogue as a collection before turning to adoption. Two of its properties bear on both. The patterns compose through information dependencies rather than fixed APIs, so a deployment instantiates a subset and binds it through shared identifiers and version references. They also divide by position relative to the trajectory: on-path patterns sit on the critical path and may gate side effects or durable state commits, while off-path patterns run asynchronously for diagnosis, evaluation, release governance, and assurance. That division organises both the trade-offs below and the rollout that follows.

4.1. Consequence trade-offs across the catalogue

Each pattern states its own Benefits and Drawbacks, but those statements only become comparable once placed on a common axis. Table 2 maps them onto the nine product quality characteristics of ISO/IEC 25010 [40]. We selected this established standard to avoid defining an ad hoc set of drivers, although mapping narrative consequences to its characteristics remains a qualitative interpretation. Privacy is represented under security/confidentiality, auditability and incident analysis under accountability and analysability, runtime cost and latency under performance efficiency, and engineering effort under maintainability. Compliance and assurance are discussed separately in Section 4.5 because they are not standalone product-quality characteristics in ISO/IEC 25010. Each symbol follows from the pattern's stated Benefits and Drawbacks under the rule given in the caption.

Two properties of the collection follow. The first is that operation is uniformly paid for in resources. Every pattern engages performance efficiency, none supports it without also taxing it, and six tax it outright. Adoption is therefore a question of which costs are worth accepting rather than whether any can be avoided, which is what makes the sequencing in Section 4.3 a decision rather than a default.

The second is that security and safety behave differently. Four patterns primarily support security and none primarily degrades it, but five record a countervailing drawback, and the reason is a tension internal to the characteristic. Accountability, the capability to trace actions uniquely to an entity, is what the evidence plane exists to provide, and the artefacts that provide it concentrate exactly the material that confidentiality would keep minimal. Safety divides differently, by what is being protected. The *Agent Gateway*, *Guardrail Enforcement Points*, and *Autonomy Controller* constrain the live trajectory at boundaries where side effects occur, whereas the *Runtime Workflow Simulator* and *Dynamic Test Case Factory* contain their own activity through sandboxed assessment and safe replay. The two are not substitutes. Off-path containment does not gate a production action, which is why the patterns that do cannot be deferred.

4.2. Implementation maturity of the catalogue

Comparable consequences do not imply comparable grounding. We therefore label each pattern with an implementation evidence maturity indicator. **Established** means that the pattern's core architectural contract is realised by released implementations from at least three independent organisations; implementations may differ in boundary coverage or deployment form but must realise the pattern's defining responsibilities. **Emerging** means implementations exist but are limited, partial, or realised as primitives inside broader tools. **Precedent-only** means the recurring need and solution structure are documented in governance frameworks, standards, or research, but no dedicated platform implementation was identified within the collection window. The indicator reflects the strength and independence of implementation evidence, not effectiveness, quality, or adoption scale.

Table 3 gives the assignment, summarising the count and character of the implementations identified. The systems themselves are named in each pattern's Known Uses, where independence should be read per organisation rather than per product. Implementation evidence is strongest for patterns that operate directly on or record the running trajectory. By contrast, we did not identify dedicated platform implementations of the *Marginal Risk Assessor* or *Assurance Evidence Factory* within the collection window. *Autonomy Controller* remains Emerging because available products more commonly support approval gating than the full combination of autonomy-posture management and boundary-specific approval decisions. This distribution informs the staged adoption guidance in Section 4.3.

[50] https://www.aisi.gov.uk/research/safety-cases-a-scalable-approach-to-frontier-ai-safety
[51] https://arxiv.org/abs/2503.04744
[52] https://arxiv.org/abs/2505.18003
[53] https://www.omg.org/spec/SACM/2.1/PDF
[54] https://www.omg.org/spec/SACM/2.1/About-SACM
[55] https://ico.org.uk/for-organisations/uk-gdpr-guidance-and-resources/artificial-intelligence/explaining-decisions-made-with-artificial-intelligence/annexe-5-argument-based-assurance-cases/

Table 2
Consequence trade-offs against the nine product quality characteristics of ISO/IEC 25010:2023. Each symbol follows from the pattern's stated Benefits and Drawbacks: + Benefits only, − Drawbacks only, ± both, blank neither. Marginal rows count symbols per characteristic. FS functional suitability, PE performance efficiency, CP compatibility, IC interaction capability, RL reliability, SE security, MN maintainability, FX flexibility, SF safety.

Pattern	FS	PE	CP	IC	RL	SE	MN	FX	SF
Runtime Workflow Simulator		−			±	±	±		+
Agent Gateway		−	+		−	±	±		±
Context Engine	+	±			−		±		
Guardrail Enforcement Points	−	−		−	−	±	+		+
Autonomy Controller	−	±		±	±	±	±		+
Trajectory Trace Plane		±	±			±	±		
Cumulative Risk Aggregator	±	±				+	±		
Dynamic Test Case Factory		−				+	±		+
Marginal Risk Assessor	±	−				+	±		
Assurance Evidence Factory	±	−				+	±		
Support (+)	1	0	1	0	0	4	1	0	4
Both (±)	3	4	1	1	2	5	9	0	1
Tax (−)	2	6	0	1	3	0	0	0	0

Table 3
Implementation-evidence maturity per pattern, with criteria in the text. Reflects the strength and independence of documented implementation evidence, not effectiveness or adoption. Named systems appear in each pattern's Known Uses.

Pattern	Maturity	Evidence basis
Runtime Workflow Simulator	Emerging	Sandboxed-execution primitives and research record-replay. No replay-and-compare harness identified as a product
Agent Gateway	Established	Gateway implementations from four independent organisations
Context Engine	Emerging	Partial context-assembly primitives inside agent frameworks, typically reduction over a single context source
Guardrail Enforcement Points	Established	Inline policy enforcement from three independent organisations, each covering a subset of the pattern's boundaries
Autonomy Controller	Emerging	Approval gating is available, but integrated autonomy-posture control remains limited
Trajectory Trace Plane	Established	Tracing implementations from six independent organisations, with draft interoperability conventions
Cumulative Risk Aggregator	Emerging	Trace-linked scoring and evaluation primitives in two products, with research on uncertainty accumulation
Dynamic Test Case Factory	Established	Trace-to-dataset workflows from four independent organisations
Marginal Risk Assessor	Precedent-only	Governance precedents defining baseline-relative risk. No platform implementation identified
Assurance Evidence Factory	Precedent-only	Governance, standards, and research precedents for assurance packaging. No platform implementation identified

4.3. For engineering leads

Because every pattern costs resources, rollout is a sequencing problem rather than a coverage exercise. A pragmatic order establishes the on-path surface first, then adds off-path learning loops, and leaves change governance last.

Increment 1, the on-path surface. Deploy the *Agent Gateway*, *Context Engine*, *Guardrail Enforcement Points*, and *Autonomy Controller*, with a minimal *Trajectory Trace Plane* for attribution. High-impact actions are then gated before effects occur, per-step context is assembled under a governed budget, and boundary outcomes are attributable through run and step identifiers bound to the policy and configuration references in force. These patterns provide the most direct live-run control and attribution capabilities, but adoption should still reflect domain risk, latency tolerance, and organisational capacity.

Increment 2, off-path learning loops. Add the *Runtime Workflow Simulator*, shadow by default and pre-flight only where risk warrants synchronous gating, together with the *Dynamic Test Case Factory* and the *Cumulative Risk Aggregator*. Recurring failures become trace-linked regression cases, and windowed rollups replace full-trace scans for routine triage. Apart from the simulator's pre-flight variant, none of this sits on the critical path, so the increment extends what increment 1 provides rather than substituting for it.

Increment 3, change governance. Add the *Marginal Risk Assessor* and the *Assurance Evidence Factory*. Releases then carry baseline-relative reports with a declared scope and window, and assurance reviews rely on claim-scoped evidence packs rather than ad hoc exports. We did not identify a dedicated released implementation of either pattern within the collection window, so teams may need to build or integrate these capabilities themselves.

Across all three, treat policy and configuration identifiers as first-class run metadata, limit exposure through canary or shadow cohorts, and move a check on-path only when the risk it addresses justifies the latency and availability it costs.

4.4. For evaluation and safety teams

The catalogue's value for evaluation is that it keeps conclusions scope-bounded and traceable to the run that produced them, which requires the decision surfaces to emit attributable signals in the first place. *Guardrail Enforcement Points* and the *Autonomy Controller* should therefore record outcomes against run and step identity, and the *Trajectory Trace Plane* should preserve a run representation that survives review. Given those, the *Runtime Workflow Simulator* and *Dynamic Test Case Factory* convert failures and near-misses into replayable cases, recording explicitly where replay is only partial, and the *Cumulative*

Risk Aggregator supports reasoning over slices and windows rather than individual runs. For a release decision, the *Marginal Risk Assessor* forces an explicit baseline and a declared scope and window, and narrowing the claim is the correct response when comparability is incomplete.

4.5. For governance and compliance

Those bounded claims are what compliance review consumes, and the catalogue supports reviewing them per run and reassessing them over declared windows without implying absolute guarantees. The organising discipline is to separate evidence generation from evidence packaging. Runtime, control, and evaluation components generate traces, enforcement and approval records, and evaluation outputs, while the *Assurance Evidence Factory* packages a scope-bounded claim with resolvable references to those artefacts and does nothing more. Two requirements make this work in practice. High-impact actions and elevated autonomy modes must pass through configured controls, and every decision must remain attributable through trace references and policy or configuration identifiers. Where confidentiality applies, minimised and tiered disclosure, such as redacted payloads and time-bounded access, preserves provenance for audit while limiting what must be exposed to reach it.

5. Related work

Architectural patterns capture recurring system-level problems and reusable solutions, typically described in terms of interacting components and their trade-offs [41,42]. We use patterns in this architectural sense, but scope them to *platform-level* building blocks for operating FM agents in production. Accordingly, we exclude agent-internal techniques such as prompting heuristics, planner algorithms, and retrieval algorithms. A pattern is in-scope only when it can be instantiated as a shared subsystem or boundary across multiple agent workflows, with an explicit interaction contract and platform-governed evidence linkage.

Prior pattern-oriented work around agentic systems spans multiple scopes. Capability and application design catalogues focus on constructing agent behaviour, including prompting and reasoning idioms, memory mechanisms, and task-level interaction structures [43,44]. Operational technique and tool catalogues describe practical mechanisms such as evaluation workflows, tracing, and safety controls [44].

Our catalogue targets the operational architecture for AgentOps platforms: reusable platform components, interfaces, and governed boundaries that can be instantiated across teams and services. The closest of these is the agent design pattern catalogue of Liu et al. [43], which shares our architectural framing but applies it to the agent rather than to the platform. Their patterns concern how an agent is designed and constructed, whereas ours concern the platform contracts under which it is operated after deployment. A production system may reasonably instantiate patterns from both, and we treat them as complementary layers rather than alternatives. Where topics overlap (e.g., guardrails, evaluation, and monitoring), our contribution is to specify the *platform-level contracts* needed to operate tool-using, stateful trajectories, including where in a run controls may gate execution (especially before side effects or durable state commits), how runtimes and governance services interact through stable mediation and decision interfaces, and how operational evidence is referenced so outcomes remain attributable under evolving models, tools, and policies.

6. Limitations and threats to validity

6.1. Limitations

The catalogue is empirically grounded to different degrees across patterns. Several patterns map to operational capabilities already instantiated in widely used tooling and frameworks, including trace-based observability, mediation at external-dependency boundaries, guardrail hooks at step boundaries, and trace-to-evaluation workflows. In contrast, *Marginal Risk Assessor* and *Assurance Evidence Factory* are supported more strongly by governance frameworks and research precedents than by mature, widely adopted AgentOps platform implementations. Their known uses should therefore be interpreted as adjacent precedents rather than evidence of broad industry saturation. We include these patterns because they correspond to recurring operational needs in production agent deployments (release decisions and audit readiness) that are often addressed through ad hoc scripts and manual processes rather than platform-owned subsystems. The maturity labels summarise implementation evidence only and should not be interpreted as evidence that a pattern is effective or widely adopted.

A second limitation is that the execution, control, and evidence planes are an organisational viewpoint rather than a mandatory physical decomposition. In practice, responsibilities may be co-located with the runtime or partially internalised within an agent trajectory (e.g., in-loop self-checks or agent-generated tests), which can blur physical boundaries even when the intended responsibilities and interaction contracts remain distinct. The catalogue specifies platform-owned contracts while allowing different physical realisations.

6.2. Threats to validity

Construct and coding validity. Pattern synthesis from heterogeneous academic and industrial sources involves judgement. Terminology and technical depth vary, which can affect pattern boundaries. We mitigate this by scoping to post-deployment platform mechanisms and specifying each pattern as an architectural contract with explicit boundaries, decision points, and evidence bindings.

Selection and process validity. Corpus construction may bias which mechanisms appear recurrent. We anchored on two broad-coverage reviews and extended with a targeted academic search, which may inherit anchor coverage gaps and miss relevant work outside our queries. The title-restricted Google Scholar query improves tractability but reduces recall.

External and temporal validity. Applicability depends on domain criticality, organisational maturity, and the feasibility of producing and protecting the required evidence (e.g., traces, configuration references, and evaluation artefacts). Tools, agent frameworks, and provider policies evolve rapidly, so documented known uses are tied to the collection window.

7. Conclusion

This paper presented an *AgentOps Architectural Pattern Catalogue* for designing and building AgentOps platforms that operate foundation-model-based agents in production. Focusing on the post-deployment operational slice, the catalogue defines ten reusable, platform-level building blocks organised across execution, control, and evidence planes. The execution and control patterns specify architectural contracts for trajectory-scoped mediation, enforcement, and approval at step boundaries in tool-mediated, stateful runs. The evidence-plane patterns specify how run-scoped artefacts and signals are captured, aggregated, compared, and packaged into reviewable evidence with explicit scope and provenance. Future work should prioritise empirical validation and operationalisation, including studying adoption variants and hybrid realisations (co-located responsibilities), strengthening trajectory-scoped evaluation and change governance, and improving policy and evidence-handling workflows for privacy, access control, and review.

The main takeaway is that operating agents in production requires three shifts from request-scoped practice. Mediation and gating belong at step boundaries immediately before external side effects and durable state commits, not only at model input and output. Operational evidence must bind to run and step identity together with the policy and configuration versions in force. Release and assurance claims must be baseline-relative and scope-bounded rather than absolute.

CRediT authorship contribution statement

Boming Xia: Writing – review & editing, Writing – original draft, Methodology, Conceptualization. **Yue Liu:** Writing – review & editing, Writing – original draft, Conceptualization. **Qinghua Lu:** Writing – review & editing, Supervision, Conceptualization. **Liming Zhu:** Writing – review & editing, Writing – original draft, Conceptualization. **Dino Sejdinovic:** Writing – review & editing, Supervision, Conceptualization. **Xiwei Xu:** Writing – review & editing, Supervision, Conceptualization.

Declaration of Generative AI and AI-assisted technologies in the writing process

During the preparation of this manuscript, the authors used *ChatGPT (OpenAI)* to assist with writing tasks such as language polishing, tightening of phrasing, and consistency checking. This use was limited to supporting the writing process only. After using this tool, the authors carefully reviewed, revised, and edited the text, and take full responsibility for all content of the published article.

Funding

This work was supported by the Responsible AI Research (RAIR) Centre, Adelaide, Australia.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

No data was used for the research described in the article.

References

[1] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, et al., A survey on large language model based autonomous agents, Front. Comput. Sci. 18 (6) (2024) 186345.

[2] Z. Xi, W. Chen, X. Guo, W. He, Y. Ding, B. Hong, M. Zhang, J. Wang, S. Jin, E. Zhou, et al., The rise and potential of large language model based agents: A survey, Sci. China Inf. Sci. 68 (2) (2025) 121101.

[3] Y. Song, G. Wang, S. Li, B.Y. Lin, The good, the bad, and the greedy: Evaluation of LLMs should not ignore non-determinism, in: Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers), 2025, pp. 4195–4206.

[4] Google, Agent development kit (ADK) documentation, 2025, URL: https://google.github.io/adk-docs/.

[5] G. Segato, Building AI products in the probabilistic era, 2025, Accessed via giansegato.com, https://giansegato.com/essays/probabilistic-era.

[6] D. Moshkovich, S. Zeltyn, Taming uncertainty via automation: Observing, analyzing, and optimizing agentic AI systems, 2025, arXiv preprint arXiv:2507.11277.

[7] L. Dong, Q. Lu, L. Zhu, AgentOps: Enabling observability of LLM agents, 2024, arXiv preprint arXiv:2411.05285.

[8] A. Gulli, L. Nigam, J. Wiesinger, V. Vuskovic, I. Sigler, I. Nardini, N. Stroppa, S. Kartakis, N. Saribekyan, A. Bount, A. Nawalgaria, S. Johnson, H. Chinoy, M. Lanning, Agents Companion, Technical Report, Google, 2025, URL: https://www.kaggle.com/whitepaper-agent-companion.

[9] Z. Wang, J. Li, Q. Zhou, H. Si, Y. Liu, J. Li, G. Xie, F. Sun, D. Pei, C. Pei, A survey on AgentOps: Categorization, challenges, and future directions, 2025, arXiv preprint arXiv:2508.02121.

[10] L. Bass, I. Weber, L. Zhu, DevOps: A Software Architect's Perspective, Addison-Wesley Professional, 2015.

[11] M. Zarour, H. Alzabut, K.T. Al-Sarayreh, MLOps best practices, challenges and maturity models: A systematic literature review, Inf. Softw. Technol. 183 (2025) 107733.

[12] C.K. Tantithamthavorn, F. Palomba, F. Khomh, J.J. Chua, MLOps, LLMOps, FMOps, and beyond, IEEE Softw. 42 (1) (2024) 26–32.

[13] Microsoft Azure Blog, Achieve generative AI operational excellence with the LLMOps maturity model, 2024, URL: https://azure.microsoft.com/en-us/blog/achieve-generative-ai-operational-excellence-with-the-llmops-maturity-model/.

[14] L. Bass, P. Clements, R. Kazman, Software Architecture in Practice, Addison-Wesley Professional, 2021.

[15] D. Kreutz, F.M. Ramos, P.E. Verissimo, C.E. Rothenberg, S. Azodolmolky, S. Uhlig, Software-defined networking: A comprehensive survey, Proc. IEEE 103 (1) (2014) 14–76.

[16] Amazon Web Services, Saas architecture fundamentals, 2022, https://docs.aws.amazon.com/pdfs/whitepapers/latest/saas-architecture-fundamentals/saas-architecture-fundamentals.pdf.

[17] V. Garousi, M. Felderer, M.V. Mäntylä, Guidelines for including grey literature and conducting multivocal literature reviews in software engineering, Inf. Softw. Technol. 106 (2019) 101–121, http://dx.doi.org/10.1016/j.infsof.2018.09.006.

[18] B. Kitchenham, O.P. Brereton, D. Budgen, M. Turner, J. Bailey, S. Linkman, Systematic literature reviews in software engineering–A systematic literature review, Inf. Softw. Technol. 51 (1) (2009) 7–15.

[19] B. Xia, Q. Lu, L. Zhu, Z. Xing, D. Zhao, H. Zhang, Evaluation-driven development of LLM agents: A process model and reference architecture, 2024, arXiv preprint arXiv:2411.13768.

[20] M.A. Ferrag, N. Tihanyi, D. Hamouda, L. Maglaras, M. Debbah, From prompt injections to protocol exploits: Threats in LLM-powered AI agents workflows, 2025, arXiv preprint arXiv:2506.23260.

[21] E. Feng, W. Zhou, Z. Liu, L. Chen, Y. Dong, C. Zhang, Y. Zhao, D. Du, Z. Hua, Y. Xia, et al., Get experience from practice: LLM agents with record & replay, 2025, arXiv preprint arXiv:2505.17716.

[22] Y. Yan, S. Wang, J. Du, Y. Yang, Y. Shan, Q. Qiu, X. Jia, X. Wang, X. Yuan, X. Han, et al., MCPWorld: A unified benchmarking testbed for API, GUI, and hybrid computer use agents, 2025, arXiv preprint arXiv:2506.07672.

[23] V. Dongre, R.A. Rossi, V.D. Lai, D.S. Yoon, D. Hakkani-Tür, T. Bui, Drift no more? Context equilibria in multi-turn LLM interactions, 2025, arXiv preprint arXiv:2510.07777.

[24] Y.P. Ji, Context engineering for AI agents: Lessons from building manus, 2025, https://manus.im/blog/Context-Engineering-for-AI-Agents-Lessons-from-Building-Manus.

[25] LangChain, Context engineering for agents, 2025, https://blog.langchain.com/context-engineering-for-agents/.

[26] M. Shamsujjoha, Q. Lu, D. Zhao, L. Zhu, Swiss cheese model for AI safety: A taxonomy and reference architecture for multi-layered guardrails of foundation model based agents, in: 2025 IEEE 22nd International Conference on Software Architecture, ICSA, IEEE, 2025, pp. 37–48.

[27] V.C. Hu, D. Ferraiolo, R. Kuhn, A. Schnitzer, K. Sandlin, R. Miller, K. Scarfone, et al., Guide to attribute based access control (ABAC) definition and considerations, NIST Spec. Publ. 800 (162) (2014) 1–54.

[28] T. Rebedea, R. Dinu, M.N. Sreedhar, C. Parisien, J. Cohen, Nemo guardrails: A toolkit for controllable and safe LLM applications with programmable rails, in: Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: System Demonstrations, 2023, pp. 431–445.

[29] H. Wang, C.M. Poskitt, J. Sun, Agentspec: Customizable runtime enforcement for safe and reliable LLM agents, 2025, arXiv preprint arXiv:2503.18666.

[30] M. Srikumar, J. Pratt, K. Chmielinski, C. Ashurst, C. Bakalar, W. Bartholomew, R. Bommasani, P. Cihon, R. Crootof, M. Hoffmann, et al., Prioritizing real-time failure detection in AI agents, 2025, Partnership on AI.

[31] Q. Zhao, D. Li, Y. Liu, W. Cheng, Y. Sun, M. Oishi, T. Osaki, K. Matsuda, H. Yao, C. Zhao, et al., Uncertainty propagation on LLM agent, in: Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), 2025, pp. 6064–6073.

[32] J. Duan, J. Diffenderfer, S. Madireddy, T. Chen, B. Kailkhura, K. Xu, UProp: Investigating the uncertainty propagation of LLMs in multi-step agentic decision-making, 2025, arXiv preprint arXiv:2506.17419.

[33] L. Zhang, Y. Cao, Y. Xie, F. Fang, Y. Li, Dynamic evaluation with cognitive reasoning for multi-turn safety of large language models, in: Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), 2025, pp. 19588–19608.

[34] J. Alaga, M. Chen, Marginal risk relative to what? Distinguishing baselines in AI risk management, in: ICML Workshop on Technical AI Governance, TAIG, 2025.

[35] J. Chen, S. Ma, Q. Lu, S.U. Lee, L. Zhu, MARIA: A framework for marginal risk assessment without ground truth in AI systems, 2025, arXiv preprint arXiv:2510.27163.

[36] M. Maksimov, N.L. Fung, S. Kokaly, M. Chechik, Two decades of assurance case tools: A survey, in: International Conference on Computer Safety, Reliability, and Security, Springer, 2018, pp. 49–59.

[37] J. Clymer, N. Gabrieli, D. Krueger, T. Larsen, Safety cases: How to justify the safety of advanced AI systems, 2024, arXiv preprint arXiv:2403.10462.

[38] T. Korbak, J. Clymer, B. Hilton, B. Shlegeris, G. Irving, A sketch of an AI control safety case, 2025, arXiv preprint arXiv:2501.17315.

[39] J. Clymer, J. Weinbaum, R. Kirk, K. Mai, S. Zhang, X. Davies, An example safety case for safeguards against misuse, 2025, arXiv preprint arXiv:2505.18003.

[40] ISO/IEC 25010:2023, systems and software engineering — Systems and software Quality Requirements and Evaluation (square) — Product quality model, 2023, ISO/IEC 25010.

[41] P. Avgeriou, U. Zdun, Architectural patterns revisited-a pattern language, 2005.
[42] S.K. Lo, Q. Lu, L. Zhu, H.-Y. Paik, X. Xu, C. Wang, Architectural patterns for the design of federated learning systems, J. Syst. Softw. 191 (2022) 111357.
[43] Y. Liu, S.K. Lo, Q. Lu, L. Zhu, D. Zhao, X. Xu, S. Harrer, J. Whittle, Agent design pattern catalogue: A collection of architectural patterns for foundation model based agents, J. Syst. Softw. 220 (2025) 112278.
[44] A. Gulli, Agentic Design Patterns: aA Hands-On Guide to Building Intelligent Systems, Springer Nature Switzerland, 2025, URL: https://books.google.com.au/books?id=QqR20QEACAAJ.